UNIVERSIDADE SÃO FRANCISCO
ENGENHARIA DA COMPUTAÇÃO

João Pedro Cezar – RA 202600601
Anderson Gabriel Geraldi de Godoi – RA 202411989
Giovani de Salvi Piffer – RA 202319687
Eduardo Suganami de Souza – RA 202327000
Luís Inácio Araujo – RA 202330746

LABRESERVA

Sistema de reserva de laboratórios e equipamentos em C

Bragança Paulista
2026

João Pedro Cezar – RA 202600601
Anderson Gabriel Geraldi de Godoi – RA 202411989
Giovani de Salvi Piffer – RA 202319687
Eduardo Suganami de Souza – RA 202327000
Luís Inácio Araujo – RA 202330746

LABRESERVA

Sistema de reserva de laboratórios e equipamentos em C

Documentação técnica do Projeto N2 de Programação Estruturada, desenvolvido em linguagem C, apresentada no curso de Engenharia da Computação.

Professor: Pedro Fascina Casarin.

Bragança Paulista
2026

SUMÁRIO

1 Projeto e instalação..... 3

2 Dados de uma reserva..... 4

3 Regras de disponibilidade..... 5

4 Demonstração e testes..... 6

5 Gravação e recuperação..... 8

6 Comparação entre as versões..... 9

Referências..... 12

Apêndice A – Guia das funções..... 13

Apêndice B – Código-fonte integral..... 22

1 Projeto e instalação

Uma turma de robótica precisa de uma sala para 24 alunos e de seis kits Arduino. Mesmo quando há lugares suficientes, os kits podem estar em outra sala. O LabReserva reúne essas duas verificações no terminal. Para organizar a agenda, a escola oferece três turnos por dia: manhã, tarde e noite.

No primeiro uso, o sistema cadastra quatro laboratórios e seis tipos de equipamento. A professora informa data, turno e tamanho da turma. Depois de escolher uma sala livre, digita seus dados e pode solicitar materiais. Esse pedido acompanha os exemplos deste documento, desde a criação até a exclusão de uma das datas.

1.1 Execução

```
gcc -std=c11 -Wall -Wextra -pedantic reserva_labs.c -o labreserva
./labreserva
```

No Windows com GCC, use labreserva.exe como nome de saída e execute .\labreserva.exe no PowerShell. O arquivo de dados fica na pasta em que o programa é executado. Para reproduzir os IDs do ensaio, comece em uma pasta vazia. Mantenha o arquivo de uma versão anterior em outra pasta, pois a versão LAB3 exige outro formato.

1.2 Versão documentada

A análise considera o arquivo reserva_labs.c da branch main, revisão 8ffc87a. O ensaio usou o padrão C11 e os testes de terminal fornecidos no repositório. A comparação da seção 6 toma como base o arquivo de 1.437 linhas que acompanhava a primeira apresentação.

2 Dados de uma reserva

A estrutura Reserva reúne dez campos. Professor, matrícula e disciplina são três campos, embora apareçam na mesma declaração. Data guarda dia, mês e ano. O turno é representado pelos inteiros 1, 2 e 3. Cada registro vincula uma sala e pode conter até dez tipos de material.

Figura 1 – Definição de Reserva na versão LAB3

```
typedef struct {
    int           id;
    char          professor[TAM_NOME], matricula[TAM_CURTO], disciplina[TAM_NOME];
    int           idLaboratorio;
    Data          data;
    int           turno;
    int           qtdAlunos;
    int           qtdItens;
    ItemReserva   itens[MAX_ITENS];
} Reserva;
```

Fonte: reserva_labs.c, linhas 52 a 61.

A sala é identificada por idLaboratorio. Para os materiais, cada posição de itens contém o ID de um equipamento e sua quantidade. No ensaio, o ID 1 da sala identifica Informática 1 e o ID 3 do equipamento identifica o kit Arduino. Com seis kits do mesmo tipo, qtdItens vale 1.

2.1 Identificação e memória

Os três cadastros ficam em vetores dinâmicos dentro de Sistema. O cabeçalho guarda quantos registros existem e quais IDs atribuir nas próximas inclusões. Uma exclusão muda posições no vetor. O ID, por sua vez, continua identificando a mesma reserva. Ao ampliar o vetor, a função primeiro verifica o ponteiro devolvido por realloc.

3 Regras de disponibilidade

Uma sala fica disponível se sua capacidade atende à turma e se nenhuma outra reserva ocupa o mesmo laboratório, data e turno. Logo, uma aula na manhã de 17/11 pode impedir outra aula na mesma sala naquela manhã e ainda deixar a sala livre à tarde. As datas válidas vão de 2000 a 2100. Para criar uma reserva, o sistema rejeita dias anteriores ao atual.

Figura 2 – Condição que identifica uma reserva no turno pesquisado

```
int ocupaTurno(Reserva *r, Data d, int turno, int idIgnorado) {
    return r->id != idIgnorado &&
           dataInt(r->data) == dataInt(d) && r->turno == turno;
}
```

Fonte: reserva_labs.c, linhas 615 a 618.

O parâmetro idIgnorado permite editar uma reserva sem colocá-la em conflito com ela mesma. A checagem do laboratório usa essa condição junto com o ID da sala. Para equipamentos, unidadesEmUso soma o material em todas as salas no dia e turno escolhidos.

3.1 Correção do pedido

A triagem informa quando o dia ou o turno estão lotados. Se uma turma de 31 alunos exceder a capacidade de todas as salas iniciais, o sistema permite informar outro tamanho ou interromper. Se existem salas grandes o bastante, mas estão ocupadas naquele turno, também permite trocar turno ou data.

A lista de materiais é opcional. Na edição, pode-se manter os itens, refazer a seleção ou remover todos. Durante uma nova seleção, −1 restaura a lista anterior e 0 conclui a escolha. Ao salvar, o programa verifica novamente a sala e o estoque.

4 Demonstração e testes

O ensaio começou em uma pasta vazia, com a professora Ana, matrícula A01, disciplina Robótica, 24 alunos e seis kits Arduino. No menu Reservas, a opção 1 abriu o cadastro. A data foi 17/11/2027, no turno 1. A sala escolhida foi Informática 1, com 30 lugares. Ao pedir duas semanas extras, o programa mostrou as datas antes da confirmação.

Figura 3 – Prévia gerada antes de gravar o lote de três aulas

```
Datas da reserva:
17/11/2027 - Reserva original: Disponivel
24/11/2027 - Disponivel
01/12/2027 - Disponivel
3 de 3 datas disponiveis.
```

Fonte: Saída capturada na execução do commit 8ffc87a.

Uma confirmação gerou os IDs 1, 2 e 3. Em uma série com conflito apenas numa data adicional, a prévia marca a cópia indisponível e informa o motivo. A data original precisa ser válida. O limite é de 30 semanas adicionais.

Figura 4 – Saldo dos equipamentos no turno da manhã de 17 de novembro

```
=== Equipamentos disponiveis ===
ID   Nome                      Categoria           Livres/Total
1    Projetor                  Audiovisual         3/3
2    Notebook                  Informatica         15/15
3    Kit Arduino               Eletronica          4/10
4    Osciloscopio              Eletronica          4/4
5    Microscopio               Laboratorio         8/8
6    Caixa de som              Audiovisual         2/2
```

Fonte: Saída capturada na consulta de disponibilidade.

Na manhã ocupada sobraram quatro kits de dez. Uma segunda consulta, agora à tarde do mesmo dia, mostrou dez livres. O estoque total continuou igual nas duas consultas.

4.1 Edição, exclusão e relatório

Na opção 4, o pedido de Ana foi alterado de 24 para 31 alunos. Ao tentar salvar, o programa informou a capacidade de 30 lugares. As alterações foram descartadas. Em outra execução, a consulta ao ID 1 confirmou que os 24 alunos originais continuavam gravados.

A opção 5 excluiu a reserva 2, referente a 24/11, após confirmação. A busca por esse ID retornou “Reserva nao encontrada”. As aulas de 17/11 e 01/12 permaneceram na agenda. Cada uma solicitou seis kits, como se vê no relatório.

Figura 5 – Relatório depois da exclusão da segunda aula

```
=== Relatorio de uso ===
Total de reservas: 2

=== Turnos reservados por laboratorio ===
Lab Informatica 1      2 turnos
Lab Informatica 2      0 turnos
Lab Quimica            0 turnos
Lab Eletronica         0 turnos

=== Total de unidades solicitadas por equipamento ===
Projetor               0 unidades
Notebook               0 unidades
Kit Arduino            12 unidades
Osciloscopio           0 unidades
Microscopio            0 unidades
Caixa de som           0 unidades
```

Fonte: Saída capturada na execução do ensaio.

Os testes automatizados do repositório também executam o programa pelo menu. Os 22 casos passaram. Entre eles estão a recusa de datas inválidas, a lotação de todas as salas, o estoque compartilhado, a repetição com conflito, a edição em etapas e a recuperação de arquivo. Cada caso utiliza uma pasta temporária.

5 Gravação e recuperação

Ao confirmar uma alteração, salvarDados escreve o cabeçalho e os vetores em reservas_lab.tmp. O programa confere a escrita e o fechamento antes de tentar substituir reservas_lab.dat. Quando o sistema operacional impede a substituição direta, o programa move o arquivo anterior para reservas_lab.bak e tenta concluir a troca.

Se a gravação falha, o chamador recebe o resultado. Na criação de uma série, o programa recupera a contagem e o próximo ID. Na edição, repõe o registro anterior. Em uma exclusão, a função restaura o registro na posição que ocupava.

5.1 Na próxima execução

O carregamento distingue um arquivo ausente de uma falha de abertura. Se faltar o .dat, tenta restaurar o .bak. Quando há um arquivo, verifica a assinatura LAB3 e o tamanho esperado, calculado a partir do cabeçalho e dos tipos de registro. Antes de aceitar os dados, confere limites de contagens, IDs seguintes e campos das reservas.

5.2 Alcance da solução

O formato armazena estruturas C em binário. A revisão melhora a resistência a falhas de gravação, mas o programa ainda foi concebido para uso local. Duas instâncias que escrevam simultaneamente na mesma pasta não dispõem de um mecanismo de bloqueio. As quantidades do relatório descrevem o total solicitado nas reservas que ainda existem.

6 Comparação entre as versões

Agendamento. Início e fim em minutos, entre 7h e 23h. Três turnos fixos: manhã, tarde e noite. A sala fica ocupada no turno inteiro. A data de hoje é aceita, independentemente da hora.

Estrutura Reserva. 12 campos, com inicioMin, fimMin e status. 10 campos. turno substitui os dois horários e o status é removido. O enum StatusReserva e as constantes de horário saem do código.

Fluxo de cadastro. O responsável era informado antes da seleção de horário e sala. O programa verifica data, turno, tamanho da turma e sala antes de pedir professor, matrícula, disciplina e materiais. É possível voltar e corrigir etapas.

Disponibilidade. A filtragem por capacidade e a checagem de horários ficavam distribuídas entre as opções. Uma regra comum verifica capacidade e ocupação. A interface distingue dia lotado, turno lotado, turma maior que toda a escola e salas livres pequenas demais.

Materiais. O saldo era calculado pelo pico minuto a minuto. A edição recomeçava a lista. O saldo usa o total reservado no mesmo dia e turno em todas as salas. Na edição, é possível manter, alterar, limpar ou restaurar os itens com −1. O zero conclui a seleção.

6.2 Repetição, edição e consultas

Repetição semanal. Primeiro confirmava-se a aula original. As cópias eram tentadas em seguida. A prévia lista a original e até 30 semanas extras. Indica sala ocupada, material insuficiente ou data fora do calendário. Uma confirmação grava as datas viáveis em lote. A original precisa ser válida.

Edição. Um submenu específico alterava a cópia. A data e os horários eram editados juntos. Cadastro e edição compartilham alterarDadosReserva. Data e turno podem ser corrigidos separadamente antes da validação final. O próprio ID é ignorado. Uma falha de gravação restaura o original.

Exclusão. Havia cancelamento com histórico e exclusão definitiva em opções diferentes. O menu passa a ter cinco operações. A opção 5 exclui a reserva após confirmação. Voltar com 0 descarta um rascunho. As reservas excluídas saem da agenda, do estoque comprometido e do relatório.

Consultas. A agenda filtrava reservas confirmadas por dia. A lista exibia horário e situação. A agenda agrupa os resultados em manhã, tarde e noite, inclusive turnos vazios. A lista usa data, turno e sala como ordem. A consulta de disponibilidade pede o turno. A busca por professor ou matrícula permanece.

Relatório. Contagem por status, horas por laboratório e gráficos de barras no terminal. Exibe o número de reservas existentes, os turnos reservados por sala e a soma das unidades solicitadas de cada equipamento. Saem as horas, as contagens de status e as barras.

6.3 Entrada, arquivo e manutenção

Entrada de texto. Uma linha longa podia ser truncada e aceita. lerLinha informa sucesso ou falha e os leitores repetem entradas longas. aparar usa ponteiros, strlen e memmove. Encerrar a entrada já não imprime uma promessa de salvamento.

Persistência. Assinatura LAB1 e gravação direta de reservas_lab.dat. A assinatura passa a LAB3. A escrita termina em .tmp antes da substituição do .dat. Se necessário, usa .bak durante a troca. Na abertura, pode restaurar o .bak quando falta o .dat.

Validação do arquivo. Assinatura, contagens negativas e leituras incompletas eram verificadas. Também confere tamanho exato, limites das contagens e próximos IDs, data, turno, número de alunos, itens e quantidades. Distingue arquivo ausente de erro de abertura. Não converte o arquivo LAB1.

Falhas e memória. As operações podiam anunciar sucesso mesmo após falha de escrita. salvarDados devolve o resultado. Inclusões, edições e exclusões revertem alterações quando a gravação falha. O lote recupera contagem e próximo ID. realloc usa ponteiro temporário e main inicializa Sistema com zero.

Manutenção. Código e mensagens seguiam a lógica de períodos e status. Novas funções separam seleção, repetição e restauração de registros. errno.h e string.h passam a ser usados. O repositório consultado inclui README, fluxo de validação no GitHub e 22 testes de terminal.

Referências

LABRESERVA. Código-fonte, documentação de execução e testes de terminal. GitHub, 2026. Revisão 8ffc87a9e12a353ca8ae3f7d2da4db2ac61380c6. Disponível em https://github.com/joaopedrocezar/lab-reserva-n2/tree/8ffc87a. Acesso em 12 nov. 2026.

ORIENTAÇÕES – Projeto N2 (Peso 2,0). 2026. Enunciado da atividade de Programação Estruturada.

Apêndice A – Guia das funções

Funções 1 a 10

`aparar()` (linha 77) Ajusta os limites do texto e move o trecho útil com memmove.

`lerLinha()` (linha 93) Lê e apara a linha. Devolve zero para entrada longa e descarta o excedente.

`lerTexto()` (linha 119) Repete a leitura até receber um texto com pelo menos um caractere depois de aparar os espaços.

`lerInteiro()` (linha 132) Exige um inteiro dentro dos limites enviados pela função chamadora e rejeita caracteres excedentes.

`lerConfirmacao()` (linha 149) Aceita uma única letra, s ou n, inclusive em maiúscula. Devolve 1 para a resposta afirmativa.

`diasNoMes()` (linha 172) Calcula a duração do mês. Para fevereiro, aplica a regra dos anos bissextos, inclusive a exceção dos séculos.

`dataValida()` (linha 181) Confere ano entre 2000 e 2100, mês entre 1 e 12 e dia compatível com o mês.

`dataInt()` (linha 186) Converte a data para o inteiro AAAAMMDD, o que permite comparar datas usando operadores numéricos.

`somarDias()` (linha 190) Avança a data pelo número de dias solicitado, ajustando as viradas de mês e de ano.

`nomeDiaSemana()` (linha 203) Conta os dias decorridos desde 1º de janeiro de 2000 e usa o resto da divisão por sete para escolher o nome do dia.

Funções 11 a 20

`dataJaPassou()` (linha 218) Compara somente o dia informado com o calendário local. Hoje é aceito.

`lerData()` (linha 229) Lê três inteiros separados por barras, verifica se há caracteres extras e chama dataValida.

`lerDataFutura()` (linha 246) Aceita data válida a partir de hoje. O zero volta ao passo anterior.

`nomeTurno()` (linha 271) Traduz os valores 1, 2 e 3 para manhã, tarde e noite.

`lerTurno()` (linha 280) Exibe as três opções de turno e permite interromper com zero.

`buscarLab()` (linha 292) Percorre o vetor de laboratórios e devolve a posição correspondente ao ID ou −1 quando ele não existe.

`buscarEquip()` (linha 301) Localiza a posição do equipamento pelo ID. Devolve −1 quando a busca termina sem correspondência.

`buscarReserva()` (linha 310) Procura o ID no vetor de reservas e devolve a posição atual ou −1.

`nomeLab()` (linha 319) Resolve o ID para o nome da sala. Exibe “(removido)” quando o cadastro já foi excluído.

`nomeEquip()` (linha 328) Resolve o ID para o nome do equipamento e usa “(removido)” para um item excluído.

Funções 21 a 30

`salvarDados()` (linha 340) Escreve os vetores em .tmp, tenta substituir .dat e usa .bak quando necessário. Devolve o resultado.

`carregarDados()` (linha 387) Distingue ausência de erro, tenta recuperar o backup, valida cabeçalho, tamanho e reservas e reconstrói os vetores.

`erroMemoria()` (linha 490) Informa a falha de alocação de memória e encerra a execução.

`adicionarLab()` (linha 495) Amplia o vetor com realloc, atribui o próximo ID e incrementa o número de laboratórios.

`adicionarEquip()` (linha 505) Amplia o vetor de equipamentos, atribui um ID sequencial e atualiza a contagem.

`adicionarReserva()` (linha 515) Amplia o vetor, atribui o próximo ID, insere a reserva e devolve o identificador.

`removerReserva()` (linha 528) Retira o registro de reserva da posição indicada, desloca os seguintes e devolve uma cópia para possível restauração.

`restaurarReserva()` (linha 538) Reinsere o registro de reserva na posição anterior quando a gravação da exclusão falha.

`removerLab()` (linha 546) Retira o registro de sala da posição indicada, desloca os seguintes e devolve uma cópia para possível restauração.

`restaurarLab()` (linha 556) Reinsere o registro de sala na posição anterior quando a gravação da exclusão falha.

Funções 31 a 40

`removerEquip()` (linha 564) Retira o registro de equipamento da posição indicada, desloca os seguintes e devolve uma cópia para possível restauração.

`restaurarEquip()` (linha 574) Reinsere o registro de equipamento na posição anterior quando a gravação da exclusão falha.

`desfazerInclusaoReservas()` (linha 583) Recupera a quantidade de reservas e o próximo ID anteriores ao lote.

`cadastrarDadosIniciais()` (linha 588) Insere os quatro laboratórios e os seis tipos de equipamento usados como base para a primeira execução.

`ocupaTurno()` (linha 615) Verifica mesma data e turno, descontando o ID ignorado.

`conflitoLab()` (linha 620) Procura a posição de outra reserva para a mesma sala, data e turno.

`qtdNaReserva()` (linha 630) Procura um equipamento entre os itens da reserva e devolve a quantidade solicitada ou zero.

`unidadesEmUso()` (linha 640) Soma a quantidade do equipamento em todas as reservas do dia e turno, salvo o ID ignorado. Interrompe ao superar MAX_ESTOQUE.

`unidadesLivres()` (linha 654) Subtrai do estoque total o uso agregado do turno e limita o saldo mínimo a zero.

`validarReserva()` (linha 659) Valida data, turno, turma, itens, sala, conflito e saldo antes da confirmação.

Funções 41 a 50

`reservaFutura()` (linha 709) Retorna verdadeiro para registros datados de hoje em diante.

`imprimirTitulo()` (linha 715) Imprime o título da seção entre sinais de igualdade, como aparece nos menus e nas figuras.

`imprimirCabecalhoLabs()` (linha 721) Imprime os nomes das colunas de identificação, bloco, tipo e capacidade das salas.

`imprimirLab()` (linha 725) Formata os campos de um laboratório em uma linha da listagem.

`listarLabs()` (linha 730) Percorre o cadastro e imprime todos os laboratórios, com o respectivo cabeçalho.

`laboratorioDisponivel()` (linha 743) Combina capacidade suficiente e ausência de outra reserva no turno.

`listarLabsLivres()` (linha 749) Lista e conta as salas que atendem ao dia, turno e tamanho da turma.

`listarEquips()` (linha 768) Apresenta os equipamentos cadastrados, suas categorias e o estoque total.

`listarEquipsLivres()` (linha 781) Exibe o saldo e o estoque total de cada equipamento no dia e turno.

`exibirDetalhes()` (linha 796) Apresenta responsável, matrícula, disciplina, sala, data, turno, turma e materiais.

Funções 51 a 60

`contemTexto()` (linha 816) Procura um trecho dentro de outro texto sem diferenciar maiúsculas e minúsculas na comparação por caractere.

`passaNoFiltro()` (linha 831) Seleciona por data, trecho de nome ou matrícula, ou permite todos os registros.

`vemAntes()` (linha 841) Ordena por data, turno e, em caso de empate, ID da sala.

`listarReservas()` (linha 851) Seleciona as posições que atendem ao filtro, ordena essa lista e imprime as reservas em ordem cronológica.

`labsDisponiveis()` (linha 898) Conta os laboratórios que satisfazem a regra compartilhada de disponibilidade.

`maiorCapacidade()` (linha 908) Obtém a capacidade da maior sala do cadastro.

`escolherLab()` (linha 918) Recebe o ID e verifica existência, capacidade e ocupação no turno. Zero permite voltar.

`explicarFaltaLab()` (linha 940) Explica se a turma excede a maior sala cadastrada ou a capacidade das salas livres naquele turno.

`configurarDisponibilidade()` (linha 963) Coordena a escolha progressiva de data, turno, turma e sala, permitindo voltar e corrigir etapas.

`selecionarEquips()` (linha 1040) Monta a lista por tipo e quantidade. Permite manter, limpar e restaurar a seleção anterior com −1.

Funções 61 a 70

`alterarDadosReserva()` (linha 1098) Altera um campo da cópia de trabalho e volta ao resumo. Na criação permite ajustar também as semanas.

`semanaDisponivel()` (linha 1148) Confere calendário, sala, capacidade e materiais de uma ocorrência. Pode informar o motivo de recusa.

`mostrarSemanas()` (linha 1171) Exibe a situação da data original e de cada repetição e devolve o número de datas viáveis.

`criarReserva()` (linha 1199) Filtra salas, pede responsável e materiais, exibe a prévia e grava a original e as cópias viáveis em lote.

`consultarReserva()` (linha 1266) Solicita o ID e apresenta os detalhes da reserva encontrada.

`escolherAlteravel()` (linha 1278) Busca o ID e impede edição de reservas em dias anteriores ao atual.

`editarReserva()` (linha 1291) Trabalha sobre uma cópia e valida o conjunto ao salvar. Restaura o registro original se a escrita falhar.

`excluirReserva()` (linha 1329) Confirma e remove o ID definitivamente. Se a persistência falhar, restaura o registro.

`criarLab()` (linha 1352) Lê nome, bloco, tipo e capacidade, inclui a sala no vetor e salva os dados. Reverte a alteração se a gravação falhar.

`editarLab()` (linha 1371) Atualiza a identificação e a capacidade da sala, respeitando o tamanho da maior turma com reserva futura. Reverte a alteração se a gravação falhar.

Funções 71 a 80

`excluirLab()` (linha 1407) Bloqueia a exclusão se houver reserva de hoje em diante na sala. Caso contrário, pede confirmação e remove o cadastro. Reverte a alteração se a gravação falhar.

`criarEquip()` (linha 1434) Lê o nome, a categoria e o estoque total. Inclui o equipamento e grava o cadastro. Reverte a alteração se a gravação falhar.

`editarEquip()` (linha 1452) Limita o novo estoque pelo maior total comprometido em um turno nas reservas de hoje em diante. Reverte a alteração se a gravação falhar.

`excluirEquip()` (linha 1491) Verifica se alguma reserva de hoje em diante utiliza o item antes de permitir sua remoção. Reverte a alteração se a gravação falhar.

`listarTodasReservas()` (linha 1520) Encaminha à listagem cronológica de todas as reservas existentes.

`exibirAgendaDoDia()` (linha 1526) Mostra as aulas de uma data agrupadas nos três turnos e identifica turnos vazios.

`buscarPorProfessor()` (linha 1552) Recebe um nome ou uma matrícula, inclusive um trecho, e mostra as reservas correspondentes.

`consultarDisponibilidade()` (linha 1560) Recebe data e turno para mostrar salas livres e saldo de materiais.

`exibirRelatorio()` (linha 1574) Conta reservas, turnos por laboratório e unidades solicitadas por tipo de equipamento.

`menuReservas()` (linha 1616) Encaminha para criar, listar, consultar, editar e excluir. Zero volta ao menu principal.

Funções 81 a 84

`menuLabs()` (linha 1637) Encaminha as opções de cadastro, listagem, edição e exclusão dos laboratórios.

`menuEquips()` (linha 1656) Encaminha as quatro operações sobre o cadastro de equipamentos.

`menuConsultas()` (linha 1675) Reúne agenda diária, busca por professor, consulta de disponibilidade e relatório de uso.

`main()` (linha 1694) Inicializa Sistema, carrega ou cria os dados, verifica o primeiro salvamento, abre o menu e libera os vetores.

Apêndice B – Código-fonte integral

Transcrição integral de reserva_labs.c. As indicações de linhas no apêndice A remetem ao arquivo original.

```
/*
 * LabReserva - Reserva de laboratórios e equipamentos (Projeto N2 - Linguagem C)
*/

#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define ARQUIVO_DADOS      "reservas_lab.dat"
#define ARQUIVO_TEMP       "reservas_lab.tmp"
#define ARQUIVO_BACKUP     "reservas_lab.bak"
#define ASSINATURA_ARQUIVO 0x4C414233 /* Identifica o formato LAB3 do arquivo. */

#define TAM_NOME           60
#define TAM_CURTO          20
#define MAX_ITENS          10
#define TURNO_MANHA        1
#define TURNO_TARDE        2
#define TURNO_NOITE        3
#define MAX_ID             1000000
#define MAX_ALUNOS         500
#define MAX_ESTOQUE        10000
#define MAX_SEMANAS_EXTRAS 30

#define FILTRO_TODAS     0
#define FILTRO_DATA      1
#define FILTRO_PROFESSOR 2

typedef struct {
    int dia, mes, ano;
} Data;

typedef struct {
    int idEquipamento, quantidade;
} ItemReserva;

typedef struct {
    int  id;
    char nome[TAM_NOME], bloco[TAM_CURTO], tipo[TAM_NOME];
    int  capacidade;
} Laboratorio;

typedef struct {
    int  id;
    char nome[TAM_NOME], categoria[TAM_NOME];
    int  quantidadeTotal;
} Equipamento;

typedef struct {
    int           id;
    char          professor[TAM_NOME], matricula[TAM_CURTO], disciplina[TAM_NOME];
    int           idLaboratorio;
```

```
    Data          data;
    int           turno;
    int           qtdAlunos;
    int           qtdItens;
    ItemReserva   itens[MAX_ITENS];
} Reserva;

typedef struct {
    int assinatura, proximoLab, proximoEquip, proximoReserva;
    int qtdLabs, qtdEquips, qtdReservas;
} Cabecalho;

typedef struct {
    Cabecalho    info;
    Laboratorio *labs;
    Equipamento *equips;
    Reserva     *reservas;
} Sistema;

/* Leitura e validação da entrada */

void aparar(char *texto) {
    char *inicio = texto;
    char *fim;

    while (isspace((unsigned char)*inicio)) {
        inicio++;
    }
    fim = inicio + strlen(inicio);
    while (fim > inicio && isspace((unsigned char)fim[-1])) {
        fim--;
    }
    *fim = '\0';
    memmove(texto, inicio, (size_t)(fim - inicio) + 1);
}

/* Rejeita entradas maiores que o campo e descarta o restante da linha. */
int lerLinha(char *destino, int tamanho) {
    char *quebra;

    if (fgets(destino, tamanho, stdin) == NULL) {
        printf("\nEntrada encerrada.\n");
        exit(0);
    }

    quebra = strchr(destino, '\n');
    if (quebra != NULL) {
        *quebra = '\0';
    } else {
        int c = getchar();
        if (c != '\n' && c != EOF) {
            while (c != '\n' && c != EOF) {
                c = getchar();
            }
            printf("  Texto muito longo para este campo.\n");
            return 0;
        }
    }

    aparar(destino);
    return 1;
}

void lerTexto(char *pergunta, char *destino, int tamanho) {
```

```
    while (1) {
        printf("%s", pergunta);
        if (!lerLinha(destino, tamanho)) {
            continue;
        }
        if (destino[0] != '\0') {
            return;
        }
        printf("  Este campo nao pode ficar vazio.\n");
    }
}

int lerInteiro(char *pergunta, int minimo, int maximo) {
    char linha[32], sobra;
    int valor;

    while (1) {
        printf("%s", pergunta);
        if (!lerLinha(linha, 32)) {
            continue;
        }
        if (sscanf(linha, "%9d%c", &valor, &sobra) == 1 &&
            valor >= minimo && valor <= maximo) {
            return valor;
        }
        printf("  Digite um numero entre %d e %d.\n", minimo, maximo);
    }
}

int lerConfirmacao(char *pergunta) {
    char linha[8];

    while (1) {
        printf("%s (s/n): ", pergunta);
        if (!lerLinha(linha, 8)) {
            continue;
        }
        if (linha[0] != '\0' && linha[1] == '\0') {
            int resposta = tolower((unsigned char)linha[0]);
            if (resposta == 's') {
                return 1;
            }
            if (resposta == 'n') {
                return 0;
            }
        }
        printf("  Responda com s ou n.\n");
    }
}

/* Datas e turnos */

int diasNoMes(int mes, int ano) {
    int dias[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    if (mes == 2 && ((ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0)) {
        return 29;
    }
    return dias[mes - 1];
}

int dataValida(Data d) {
    return d.ano >= 2000 && d.ano <= 2100 && d.mes >= 1 && d.mes <= 12 &&
           d.dia >= 1 && d.dia <= diasNoMes(d.mes, d.ano);
```

```
}

int dataInt(Data d) {
    return d.ano * 10000 + d.mes * 100 + d.dia;
}

Data somarDias(Data d, int dias) {
    d.dia = d.dia + dias;
    while (d.dia > diasNoMes(d.mes, d.ano)) {
        d.dia = d.dia - diasNoMes(d.mes, d.ano);
        d.mes++;
        if (d.mes > 12) {
            d.mes = 1;
            d.ano++;
        }
    }
    return d;
}

char *nomeDiaSemana(Data d) {
    char *nomes[7] = {"Sabado", "Domingo", "Segunda", "Terca", "Quarta", "Quinta", "Sexta"};
    int totalDias = d.dia - 1;

    for (int ano = 2000; ano < d.ano; ano++) {
        for (int mes = 1; mes <= 12; mes++) {
            totalDias += diasNoMes(mes, ano);
        }
    }
    for (int mes = 1; mes < d.mes; mes++) {
        totalDias += diasNoMes(mes, d.ano);
    }
    return nomes[totalDias % 7];
}

int dataJaPassou(Data d) {
    time_t agora = time(NULL);
    struct tm *t = localtime(&agora);

    if (t == NULL) {
        return 0;
    }
    Data hoje = {t->tm_mday, t->tm_mon + 1, t->tm_year + 1900};
    return dataInt(d) < dataInt(hoje);
}

void lerData(char *pergunta, Data *d) {
    char linha[32], sobra;

    while (1) {
        printf("%s", pergunta);
        if (!lerLinha(linha, 32)) {
            continue;
        }
        if (sscanf(linha, "%d/%d/%d%c", &d->dia, &d->mes, &d->ano, &sobra) == 3 &&
            dataValida(*d)) {
            return;
        }
        printf("  Data invalida. Use o formato DD/MM/AAAA.\n");
    }
}

/* A entrada da data pode ser interrompida com 0 sem perder os demais campos. */
int lerDataFutura(Data *d) {
    char linha[32], sobra;
```

```
    Data informada;

    while (1) {
        printf("Data (DD/MM/AAAA, 0 para voltar): ");
        if (!lerLinha(linha, sizeof(linha))) {
            continue;
        }
        if (strcmp(linha, "0") == 0) {
            return 0;
        }
        if (sscanf(linha, "%d/%d/%d%c", &informada.dia, &informada.mes,
                   &informada.ano, &sobra) == 3 && dataValida(informada)) {
            if (!dataJaPassou(informada)) {
                *d = informada;
                return 1;
            }
            printf("  Nao e possivel reservar em uma data que ja passou.\n");
        } else {
            printf("  Data invalida. Use o formato DD/MM/AAAA.\n");
        }
    }
}

char *nomeTurno(int turno) {
    switch (turno) {
    case TURNO_MANHA: return "Manha";
    case TURNO_TARDE: return "Tarde";
    case TURNO_NOITE: return "Noite";
    default: return "Invalido";
    }
}

int lerTurno(void) {
    printf("\nTurno:\n");
    printf("1 - Manha\n");
    printf("2 - Tarde\n");
    printf("3 - Noite\n");
    printf("0 - Cancelar\n");
    return lerInteiro("Opcao: ", 0, 3);
}

/* Busca nos vetores */

/* Os IDs permanecem fixos mesmo quando uma exclusão muda as posições nos vetores. */
int buscarLab(Sistema *s, int id) {
    for (int i = 0; i < s->info.qtdLabs; i++) {
        if (s->labs[i].id == id) {
            return i;
        }
    }
    return -1;
}

int buscarEquip(Sistema *s, int id) {
    for (int i = 0; i < s->info.qtdEquips; i++) {
        if (s->equips[i].id == id) {
            return i;
        }
    }
    return -1;
}

int buscarReserva(Sistema *s, int id) {
    for (int i = 0; i < s->info.qtdReservas; i++) {
```

```
        if (s->reservas[i].id == id) {
            return i;
        }
    }
    return -1;
}

char *nomeLab(Sistema *s, int id) {
    int posicao = buscarLab(s, id);

    if (posicao == -1) {
        return "(removido)";
    }
    return s->labs[posicao].nome;
}

char *nomeEquip(Sistema *s, int id) {
    int posicao = buscarEquip(s, id);

    if (posicao == -1) {
        return "(removido)";
    }
    return s->equips[posicao].nome;
}

/* Arquivo binário */

/* Grava primeiro no .tmp para não danificar o .dat se a escrita falhar. */
int salvarDados(Sistema *s) {
    FILE *f = fopen(ARQUIVO_TEMP, "wb");
    int ok = 1;

    if (f == NULL) {
        printf("  Nao foi possivel salvar os dados.\n");
        return 0;
    }

    if (fwrite(&s->info, sizeof(Cabecalho), 1, f) != 1 ||
        (s->info.qtdLabs > 0 &&
         fwrite(s->labs, sizeof(Laboratorio), (size_t)s->info.qtdLabs, f) != (size_t)s->info.qtdLabs) ||
        (s->info.qtdEquips > 0 &&
         fwrite(s->equips, sizeof(Equipamento), (size_t)s->info.qtdEquips, f) != (size_t)s->info.qtdEquips)
||
        (s->info.qtdReservas > 0 &&
         fwrite(s->reservas, sizeof(Reserva), (size_t)s->info.qtdReservas, f) != (size_t)s-
>info.qtdReservas)) {
        ok = 0;
    }
    if (fclose(f) != 0) {
        ok = 0;
    }
    if (!ok) {
        remove(ARQUIVO_TEMP);
        printf("  Falha ao gravar os dados.\n");
        return 0;
    }

    /* No Windows, rename não substitui o .dat existente; o .bak permite a troca. */
    if (rename(ARQUIVO_TEMP, ARQUIVO_DADOS) != 0) {
        remove(ARQUIVO_BACKUP);
        if (rename(ARQUIVO_DADOS, ARQUIVO_BACKUP) != 0) {
            remove(ARQUIVO_TEMP);
            printf("  Falha ao substituir o arquivo de dados.\n");
            return 0;
        }
        if (rename(ARQUIVO_TEMP, ARQUIVO_DADOS) != 0) {
```

```
            rename(ARQUIVO_BACKUP, ARQUIVO_DADOS);
            remove(ARQUIVO_TEMP);
            printf("  Falha ao substituir o arquivo de dados.\n");
            return 0;
        }
        remove(ARQUIVO_BACKUP);
    }
    return 1;
}

/* Retorna 0 se não houver arquivo; se estiver inválido, encerra sem sobrescrevê-lo. */
int carregarDados(Sistema *s) {
    FILE *f = fopen(ARQUIVO_DADOS, "rb");
    int ok = 1;
    long tamanhoArquivo;

    if (f == NULL) {
        if (errno != ENOENT) {
            printf("Nao foi possivel abrir o arquivo de dados.\n");
            exit(1);
        }
        /* Recupera o .bak se uma interrupção deixou o programa sem o .dat. */
        if (rename(ARQUIVO_BACKUP, ARQUIVO_DADOS) != 0) {
            if (errno != ENOENT) {
                printf("Nao foi possivel restaurar o backup.\n");
                exit(1);
            }
            return 0;
        }
        f = fopen(ARQUIVO_DADOS, "rb");
        if (f == NULL) {
            printf("Nao foi possivel recuperar o arquivo de dados.\n");
            exit(1);
        }
    }

    if (fseek(f, 0, SEEK_END) != 0) {
        ok = 0;
    }
    tamanhoArquivo = ok ? ftell(f) : -1;
    if (tamanhoArquivo < 0 || fseek(f, 0, SEEK_SET) != 0) {
        ok = 0;
    }
    if (ok && (fread(&s->info, sizeof(Cabecalho), 1, f) != 1 ||
        s->info.assinatura != ASSINATURA_ARQUIVO ||
        s->info.qtdLabs < 0 || s->info.qtdLabs > MAX_ID ||
        s->info.qtdEquips < 0 || s->info.qtdEquips > MAX_ID ||
        s->info.qtdReservas < 0 || s->info.qtdReservas > MAX_ID ||
        s->info.proximoLab < 1 || s->info.proximoEquip < 1 || s->info.proximoReserva < 1 ||
        s->info.proximoLab > MAX_ID + 1 || s->info.proximoEquip > MAX_ID + 1 ||
        s->info.proximoReserva > MAX_ID + 1)) {
        ok = 0;
    }
    if (ok) {
        size_t esperado = sizeof(Cabecalho) +
            (size_t)s->info.qtdLabs * sizeof(Laboratorio) +
            (size_t)s->info.qtdEquips * sizeof(Equipamento) +
            (size_t)s->info.qtdReservas * sizeof(Reserva);
        if ((size_t)tamanhoArquivo != esperado) {
            ok = 0;
        }
    }
    if (ok && s->info.qtdLabs > 0) {
        s->labs = malloc((size_t)s->info.qtdLabs * sizeof(Laboratorio));
```

```
        if (s->labs == NULL ||
            fread(s->labs, sizeof(Laboratorio), (size_t)s->info.qtdLabs, f) != (size_t)s->info.qtdLabs) {
            ok = 0;
        }
    }
    if (ok && s->info.qtdEquips > 0) {
        s->equips = malloc((size_t)s->info.qtdEquips * sizeof(Equipamento));
        if (s->equips == NULL ||
            fread(s->equips, sizeof(Equipamento), (size_t)s->info.qtdEquips, f) != (size_t)s-
>info.qtdEquips) {
            ok = 0;
        }
    }
    if (ok && s->info.qtdReservas > 0) {
        s->reservas = malloc((size_t)s->info.qtdReservas * sizeof(Reserva));
        if (s->reservas == NULL ||
            fread(s->reservas, sizeof(Reserva), (size_t)s->info.qtdReservas, f) != (size_t)s-
>info.qtdReservas) {
            ok = 0;
        }
    }
    if (ok) {
        for (int i = 0; i < s->info.qtdReservas; i++) {
            Reserva *r = &s->reservas[i];
            if (r->qtdItens < 0 || r->qtdItens > MAX_ITENS || !dataValida(r->data) ||
                r->turno < TURNO_MANHA || r->turno > TURNO_NOITE || r->qtdAlunos < 1 || r->qtdAlunos >
MAX_ALUNOS) {
                ok = 0;
                break;
            }
            for (int j = 0; j < r->qtdItens; j++) {
                if (r->itens[j].quantidade < 1 || r->itens[j].quantidade > MAX_ESTOQUE) {
                    ok = 0;
                    break;
                }
            }
            if (!ok) {
                break;
            }
        }
    }
    fclose(f);

    if (!ok) {
        printf("Erro: o arquivo %s esta danificado ou e de outra versao do programa.\n",
               ARQUIVO_DADOS);
        printf("Guarde uma copia do arquivo antes de tentar recupera-lo.\n");
        exit(1);
    }
    return 1;
}


void erroMemoria(void) {
    printf("Erro: memoria insuficiente.\n");
    exit(1);
}

void adicionarLab(Sistema *s, Laboratorio novo) {
    Laboratorio *aumentado = realloc(s->labs, (size_t)(s->info.qtdLabs + 1) * sizeof(Laboratorio));
    if (aumentado == NULL) {
        erroMemoria();
    }
    s->labs = aumentado;
    novo.id = s->info.proximoLab++;
    s->labs[s->info.qtdLabs++] = novo;
}
```

```
void adicionarEquip(Sistema *s, Equipamento novo) {
    Equipamento *aumentado = realloc(s->equips, (size_t)(s->info.qtdEquips + 1) * sizeof(Equipamento));
    if (aumentado == NULL) {
        erroMemoria();
    }
    s->equips = aumentado;
    novo.id = s->info.proximoEquip++;
    s->equips[s->info.qtdEquips++] = novo;
}

int adicionarReserva(Sistema *s, Reserva nova) {
    Reserva *aumentado = realloc(s->reservas, (size_t)(s->info.qtdReservas + 1) * sizeof(Reserva));
    if (aumentado == NULL) {
        erroMemoria();
    }
    s->reservas = aumentado;
    nova.id = s->info.proximoReserva++;
    s->reservas[s->info.qtdReservas++] = nova;
    return nova.id;
}

/* Se o salvamento falhar, os registros removidos voltam à mesma posição.
 * A remoção reduz a contagem, mas mantém espaço para restaurá-los. */
Reserva removerReserva(Sistema *s, int posicao) {
    Reserva removida = s->reservas[posicao];

    for (int i = posicao; i < s->info.qtdReservas - 1; i++) {
        s->reservas[i] = s->reservas[i + 1];
    }
    s->info.qtdReservas--;
    return removida;
}

void restaurarReserva(Sistema *s, int posicao, Reserva removida) {
    for (int i = s->info.qtdReservas; i > posicao; i--) {
        s->reservas[i] = s->reservas[i - 1];
    }
    s->reservas[posicao] = removida;
    s->info.qtdReservas++;
}

Laboratorio removerLab(Sistema *s, int posicao) {
    Laboratorio removido = s->labs[posicao];

    for (int i = posicao; i < s->info.qtdLabs - 1; i++) {
        s->labs[i] = s->labs[i + 1];
    }
    s->info.qtdLabs--;
    return removido;
}

void restaurarLab(Sistema *s, int posicao, Laboratorio removido) {
    for (int i = s->info.qtdLabs; i > posicao; i--) {
        s->labs[i] = s->labs[i - 1];
    }
    s->labs[posicao] = removido;
    s->info.qtdLabs++;
}

Equipamento removerEquip(Sistema *s, int posicao) {
    Equipamento removido = s->equips[posicao];

    for (int i = posicao; i < s->info.qtdEquips - 1; i++) {
```

```
        s->equips[i] = s->equips[i + 1];
    }
    s->info.qtdEquips--;
    return removido;
}

void restaurarEquip(Sistema *s, int posicao, Equipamento removido) {
    for (int i = s->info.qtdEquips; i > posicao; i--) {
        s->equips[i] = s->equips[i - 1];
    }
    s->equips[posicao] = removido;
    s->info.qtdEquips++;
}

/* Como novas reservas entram no fim, basta restaurar a contagem e o próximo ID. */
void desfazerInclusaoReservas(Sistema *s, int qtdAnterior, int proximoAnterior) {
    s->info.qtdReservas = qtdAnterior;
    s->info.proximoReserva = proximoAnterior;
}

void cadastrarDadosIniciais(Sistema *s) {
    Laboratorio labs[4] = {
        {0, "Lab Informatica 1", "A", "Informatica", 30},
        {0, "Lab Informatica 2", "A", "Informatica", 20},
        {0, "Lab Quimica",       "B", "Quimica",     25},
        {0, "Lab Eletronica",    "C", "Eletronica",  16}
    };
    Equipamento equips[6] = {
        {0, "Projetor",     "Audiovisual", 3},
        {0, "Notebook",     "Informatica", 15},
        {0, "Kit Arduino",  "Eletronica",  10},
        {0, "Osciloscopio", "Eletronica",  4},
        {0, "Microscopio",  "Laboratorio", 8},
        {0, "Caixa de som", "Audiovisual", 2}
    };

    for (int i = 0; i < 4; i++) {
        adicionarLab(s, labs[i]);
    }
    for (int i = 0; i < 6; i++) {
        adicionarEquip(s, equips[i]);
    }
}

/* Regras de negócio */

/* O idIgnorado evita que uma reserva em edição conflite com ela mesma. */
int ocupaTurno(Reserva *r, Data d, int turno, int idIgnorado) {
    return r->id != idIgnorado &&
           dataInt(r->data) == dataInt(d) && r->turno == turno;
}

int conflitoLab(Sistema *s, int idLab, Data d, int turno, int idIgnorado) {
    for (int i = 0; i < s->info.qtdReservas; i++) {
        if (s->reservas[i].idLaboratorio == idLab &&
            ocupaTurno(&s->reservas[i], d, turno, idIgnorado)) {
            return i;
        }
    }
    return -1;
}

int qtdNaReserva(Reserva *r, int idEquip) {
    for (int i = 0; i < r->qtdItens; i++) {
```

```
        if (r->itens[i].idEquipamento == idEquip) {
            return r->itens[i].quantidade;
        }
    }
    return 0;
}

/* Soma as unidades reservadas no mesmo dia e turno, em qualquer laboratório. */
int unidadesEmUso(Sistema *s, int idEquip, Data d, int turno, int idIgnorado) {
    int uso = 0;

    for (int i = 0; i < s->info.qtdReservas; i++) {
        if (ocupaTurno(&s->reservas[i], d, turno, idIgnorado)) {
            uso += qtdNaReserva(&s->reservas[i], idEquip);
            if (uso > MAX_ESTOQUE) {
                return MAX_ESTOQUE + 1;
            }
        }
    }
    return uso;
}

int unidadesLivres(Sistema *s, Equipamento *e, Data d, int turno, int idIgnorado) {
    int livres = e->quantidadeTotal - unidadesEmUso(s, e->id, d, turno, idIgnorado);
    return livres > 0 ? livres : 0;
}

int validarReserva(Sistema *s, Reserva *r, int idIgnorado) {
    int posicaoLab = buscarLab(s, r->idLaboratorio);
    int valido = 1;

    if (!dataValida(r->data) || r->turno < TURNO_MANHA || r->turno > TURNO_NOITE) {
        printf("  - Data ou turno invalido.\n");
        return 0;
    }
    if (dataJaPassou(r->data)) {
        printf("  - A data da reserva ja passou.\n");
        valido = 0;
    }
    if (r->qtdItens < 0 || r->qtdItens > MAX_ITENS ||
        r->qtdAlunos < 1 || r->qtdAlunos > MAX_ALUNOS) {
        printf("  - Quantidade de alunos ou itens invalida.\n");
        return 0;
    }
    if (posicaoLab == -1) {
        printf("  - O laboratorio escolhido nao existe mais.\n");
        return 0;
    }
    if (r->qtdAlunos > s->labs[posicaoLab].capacidade) {
        printf("  - %s comporta %d alunos, mas a turma tem %d.\n",
               s->labs[posicaoLab].nome, s->labs[posicaoLab].capacidade, r->qtdAlunos);
        valido = 0;
    }
    int posicaoConflito = conflitoLab(s, r->idLaboratorio, r->data, r->turno, idIgnorado);
    if (posicaoConflito != -1) {
        Reserva *outra = &s->reservas[posicaoConflito];
        printf("  - %s ja esta reservado nesse dia e turno para %s (reserva %d).\n",
               s->labs[posicaoLab].nome, outra->professor, outra->id);
        valido = 0;
    }
    for (int i = 0; i < r->qtdItens; i++) {
        int posicaoEquip = buscarEquip(s, r->itens[i].idEquipamento);
        if (posicaoEquip == -1) {
            printf("  - Um dos equipamentos nao existe mais.\n");
```

```
            valido = 0;
            continue;
        }
        int livres = unidadesLivres(s, &s->equips[posicaoEquip], r->data, r->turno, idIgnorado);
        if (r->itens[i].quantidade < 1 || r->itens[i].quantidade > livres) {
            printf("  - %s: pedidas %d unidades, disponiveis %d.\n",
                   s->equips[posicaoEquip].nome, r->itens[i].quantidade, livres);
            valido = 0;
        }
    }
    return valido;
}

int reservaFutura(Reserva *r) {
    return !dataJaPassou(r->data);
}

/* Exibição */

void imprimirTitulo(char *titulo) {
    printf("\n=== %s ===\n", titulo);
}



void imprimirCabecalhoLabs(void) {
    printf("%-4s %-28s %-7s %-20s %s\n", "ID", "Nome", "Bloco", "Tipo", "Capacidade");
}

void imprimirLab(Laboratorio *l) {
    printf("%-4d %-28.28s %-7.7s %-20.20s %d\n",
           l->id, l->nome, l->bloco, l->tipo, l->capacidade);
}

void listarLabs(Sistema *s) {
    imprimirTitulo("Laboratorios");
    if (s->info.qtdLabs == 0) {
        printf("Nenhum laboratorio cadastrado.\n");
        return;
    }
    imprimirCabecalhoLabs();
    for (int i = 0; i < s->info.qtdLabs; i++) {
        imprimirLab(&s->labs[i]);
    }
}

/* Uma unica regra de capacidade e ocupacao para listagens e escolhas. */
int laboratorioDisponivel(Sistema *s, Laboratorio *l, Data d, int turno,
                          int alunos, int idIgnorado) {
    return l->capacidade >= alunos &&
           conflitoLab(s, l->id, d, turno, idIgnorado) == -1;
}

int listarLabsLivres(Sistema *s, Data d, int turno, int alunos, int idIgnorado) {
    int achados = 0;

    imprimirTitulo("Laboratorios disponiveis");
    for (int i = 0; i < s->info.qtdLabs; i++) {
        if (laboratorioDisponivel(s, &s->labs[i], d, turno, alunos, idIgnorado)) {
            if (achados == 0) {
                imprimirCabecalhoLabs();
            }
            imprimirLab(&s->labs[i]);
            achados++;
```

```
        }
    }
    if (achados == 0) {
        printf("Nenhum laboratorio disponivel para esses criterios.\n");
    }
    return achados;
}

void listarEquips(Sistema *s) {
    imprimirTitulo("Equipamentos");
    if (s->info.qtdEquips == 0) {
        printf("Nenhum equipamento cadastrado.\n");
        return;
    }
    printf("%-4s %-28s %-20s %s\n", "ID", "Nome", "Categoria", "Quantidade");
    for (int i = 0; i < s->info.qtdEquips; i++) {
        printf("%-4d %-28.28s %-20.20s %d\n", s->equips[i].id, s->equips[i].nome,
               s->equips[i].categoria, s->equips[i].quantidadeTotal);
    }
}

void listarEquipsLivres(Sistema *s, Data d, int turno, int idIgnorado) {
    imprimirTitulo("Equipamentos disponiveis");
    if (s->info.qtdEquips == 0) {
        printf("Nenhum equipamento cadastrado.\n");
        return;
    }
    printf("%-4s %-28s %-20s %s\n", "ID", "Nome", "Categoria", "Livres/Total");
    for (int i = 0; i < s->info.qtdEquips; i++) {
        printf("%-4d %-28.28s %-20.20s %d/%d\n", s->equips[i].id, s->equips[i].nome,
               s->equips[i].categoria,
               unidadesLivres(s, &s->equips[i], d, turno, idIgnorado),
               s->equips[i].quantidadeTotal);
    }
}

void exibirDetalhes(Sistema *s, Reserva *r) {
    printf("  Professor : %s (matricula %s)\n", r->professor, r->matricula);
    printf("  Disciplina: %s\n", r->disciplina);
    printf("  Lab       : %s\n", nomeLab(s, r->idLaboratorio));
    printf("  Data      : %s, %02d/%02d/%04d\n",
           nomeDiaSemana(r->data), r->data.dia, r->data.mes, r->data.ano);
    printf("  Turno     : %s\n", nomeTurno(r->turno));
    printf("  Alunos    : %d\n", r->qtdAlunos);
    printf("  Equipamentos:");
    if (r->qtdItens == 0) {
        printf(" nenhum");
    }
    printf("\n");
    for (int i = 0; i < r->qtdItens; i++) {
        printf("    - %dx %s\n", r->itens[i].quantidade,
               nomeEquip(s, r->itens[i].idEquipamento));
    }
}

/* Compara letras ASCII sem diferenciar maiúsculas e minúsculas; não trata acentos. */
int contemTexto(char *texto, char *procurado) {
    for (int i = 0; texto[i] != '\0'; i++) {
        int j = 0;

        while (procurado[j] != '\0' &&
               tolower((unsigned char)texto[i + j]) == tolower((unsigned char)procurado[j])) {
            j++;
        }
```

```
        if (procurado[j] == '\0') {
            return 1;
        }
    }
    return 0;
}

int passaNoFiltro(Reserva *r, int filtro, Data d, char *texto) {
    if (filtro == FILTRO_DATA) {
        return dataInt(r->data) == dataInt(d);
    }
    if (filtro == FILTRO_PROFESSOR) {
        return contemTexto(r->professor, texto) || contemTexto(r->matricula, texto);
    }
    return 1;
}

int vemAntes(Reserva *a, Reserva *b) {
    if (dataInt(a->data) != dataInt(b->data)) {
        return dataInt(a->data) < dataInt(b->data);
    }
    if (a->turno != b->turno) {
        return a->turno < b->turno;
    }
    return a->idLaboratorio < b->idLaboratorio;
}

void listarReservas(Sistema *s, char *titulo, int filtro, Data d, char *texto) {
    int *lista = malloc((size_t)(s->info.qtdReservas + 1) * sizeof(int));
    int quantidade = 0;

    imprimirTitulo(titulo);
    if (lista == NULL) {
        printf("Memoria insuficiente para montar a lista.\n");
        return;
    }
    for (int i = 0; i < s->info.qtdReservas; i++) {
        if (passaNoFiltro(&s->reservas[i], filtro, d, texto)) {
            lista[quantidade] = i;
            quantidade++;
        }
    }
    if (quantidade == 0) {
        printf("Nenhuma reserva encontrada.\n");
        free(lista);
        return;
    }

    /* Ordena os índices, sem mudar a ordem das reservas armazenadas. */
    for (int i = 0; i < quantidade - 1; i++) {
        for (int j = 0; j < quantidade - 1 - i; j++) {
            if (vemAntes(&s->reservas[lista[j + 1]], &s->reservas[lista[j]])) {
                int temporario = lista[j];
                lista[j] = lista[j + 1];
                lista[j + 1] = temporario;
            }
        }
    }

    printf("%-4s %-10s %-8s %-20s %-20s %-18s %s\n", "ID", "Data", "Turno",
           "Laboratorio", "Professor", "Disciplina", "Alunos");
    for (int i = 0; i < quantidade; i++) {
        Reserva *r = &s->reservas[lista[i]];
```

```
        printf("%-4d %02d/%02d/%04d %-8s %-20.20s %-20.20s %-18.18s %d\n",
               r->id, r->data.dia, r->data.mes, r->data.ano,
               nomeTurno(r->turno), nomeLab(s, r->idLaboratorio), r->professor,
               r->disciplina, r->qtdAlunos);
    }
    free(lista);
}

/* CRUD de reservas */

int labsDisponiveis(Sistema *s, Data d, int turno, int alunos, int idIgnorado) {
    int total = 0;
    for (int i = 0; i < s->info.qtdLabs; i++) {
        if (laboratorioDisponivel(s, &s->labs[i], d, turno, alunos, idIgnorado)) {
            total++;
        }
    }
    return total;
}

int maiorCapacidade(Sistema *s) {
    int maior = 0;
    for (int i = 0; i < s->info.qtdLabs; i++) {
        if (s->labs[i].capacidade > maior) {
            maior = s->labs[i].capacidade;
        }
    }
    return maior;
}

int escolherLab(Sistema *s, Reserva *r, int idIgnorado) {
    while (1) {
        int id = lerInteiro("ID do laboratorio (0 para voltar): ", 0, MAX_ID);
        int posicao;

        if (id == 0) {
            return 0;
        }
        posicao = buscarLab(s, id);
        if (posicao == -1) {
            printf("  Laboratorio nao encontrado.\n");
        } else if (s->labs[posicao].capacidade < r->qtdAlunos) {
            printf("  Este laboratorio comporta apenas %d alunos.\n", s->labs[posicao].capacidade);
        } else if (conflitoLab(s, id, r->data, r->turno, idIgnorado) != -1) {
            printf("  Este laboratorio ja esta ocupado nesse turno.\n");
        } else {
            r->idLaboratorio = id;
            return 1;
        }
    }
}

void explicarFaltaLab(Sistema *s, Data d, int turno, int alunos, int idIgnorado) {
    int maiorTotal = maiorCapacidade(s);
    int maiorLivre = 0;

    for (int i = 0; i < s->info.qtdLabs; i++) {
        Laboratorio *l = &s->labs[i];
        if (laboratorioDisponivel(s, l, d, turno, 0, idIgnorado) &&
            l->capacidade > maiorLivre) {
            maiorLivre = l->capacidade;
        }
    }
    if (alunos > maiorTotal) {
```

```
        printf("Nenhum laboratorio comporta %d alunos. Maior capacidade: %d.\n",
               alunos, maiorTotal);
    } else if (maiorLivre == 0) {
        printf("Todos os laboratorios estao ocupados nesse dia e turno.\n");
    } else {
        printf("Os laboratorios livres comportam no maximo %d alunos.\n", maiorLivre);
        printf("Os laboratorios que comportam %d alunos estao ocupados.\n", alunos);
    }
}

/* Valida a disponibilidade enquanto o professor escolhe data, turno e turma. */
int configurarDisponibilidade(Sistema *s, Reserva *r, int idIgnorado) {
    int etapa = 0;

    while (1) {
        if (etapa == 0) {
            int existe = 0;
            if (!lerDataFutura(&r->data)) {
                return 0;
            }
            for (int turno = TURNO_MANHA; turno <= TURNO_NOITE; turno++) {
                if (labsDisponiveis(s, r->data, turno, 0, idIgnorado) > 0) {
                    existe = 1;
                    break;
                }
            }
            if (!existe) {
                printf("Nenhum laboratorio livre nessa data, em nenhum turno.\n");
                printf("1 - Escolher outra data\n0 - Cancelar\n");
                if (lerInteiro("Opcao: ", 0, 1) == 0) {
                    return 0;
                }
                continue;
            }
            etapa = 1;
        }
        if (etapa == 1) {
            r->turno = lerTurno();
            if (r->turno == 0) {
                return 0;
            }
            if (labsDisponiveis(s, r->data, r->turno, 0, idIgnorado) == 0) {
                printf("Todos os laboratorios estao ocupados nesse turno.\n");
                printf("1 - Outro turno\n2 - Outra data\n0 - Cancelar\n");
                int opcao = lerInteiro("Opcao: ", 0, 2);
                if (opcao == 0) {
                    return 0;
                }
                etapa = opcao == 2 ? 0 : 1;
                continue;
            }
            etapa = 2;
        }
        if (etapa == 2) {
            r->qtdAlunos = lerInteiro("Quantidade de alunos: ", 1, MAX_ALUNOS);
            if (labsDisponiveis(s, r->data, r->turno, r->qtdAlunos, idIgnorado) == 0) {
                explicarFaltaLab(s, r->data, r->turno, r->qtdAlunos, idIgnorado);
                if (r->qtdAlunos > maiorCapacidade(s)) {
                    printf("1 - Alterar alunos\n0 - Cancelar\n");
                    if (lerInteiro("Opcao: ", 0, 1) == 0) {
                        return 0;
                    }
                    etapa = 2;
                } else {
```

```
                    printf("1 - Alterar alunos\n2 - Alterar turno\n3 - Alterar data\n0 - Cancelar\n");
                    int opcao = lerInteiro("Opcao: ", 0, 3);
                    if (opcao == 0) {
                        return 0;
                    }
                    etapa = opcao == 1 ? 2 : (opcao == 2 ? 1 : 0);
                }
                continue;
            }
            etapa = 3;
        }
        listarLabsLivres(s, r->data, r->turno, r->qtdAlunos, idIgnorado);
        if (escolherLab(s, r, idIgnorado)) {
            return 1;
        }
        printf("1 - Alterar alunos\n2 - Alterar turno\n3 - Alterar data\n0 - Cancelar\n");
        int opcao = lerInteiro("Opcao: ", 0, 3);
        if (opcao == 0) {
            return 0;
        }
        etapa = opcao == 1 ? 2 : (opcao == 2 ? 1 : 0);
    }
}

void selecionarEquips(Sistema *s, Reserva *r, int idIgnorado) {
    if (s->info.qtdEquips == 0) {
        printf("Nenhum equipamento cadastrado.\n");
        return;
    }
    if (r->qtdItens > 0) {
        printf("\nEquipamentos selecionados:\n");
        for (int i = 0; i < r->qtdItens; i++) {
            printf("  %dx %s\n", r->itens[i].quantidade,
                   nomeEquip(s, r->itens[i].idEquipamento));
        }
        printf("1 - Alterar equipamentos\n2 - Remover todos\n0 - Manter atuais\n");
        int opcao = lerInteiro("Opcao: ", 0, 2);
        if (opcao == 0) return;
        if (opcao == 2) {
            r->qtdItens = 0;
            return;
        }
    } else if (!lerConfirmacao("Deseja reservar equipamentos?")) {
        return;
    }

    Reserva anterior = *r;
    r->qtdItens = 0;
    listarEquipsLivres(s, r->data, r->turno, idIgnorado);

    while (r->qtdItens < MAX_ITENS) {
        char pergunta[64];
        int indice = 0;
        int id = lerInteiro("ID do equipamento (-1 cancelar, 0 concluir): ", -1, MAX_ID);
        if (id == -1) {
            *r = anterior;
            printf("Alteracao de equipamentos descartada.\n");
            return;
        }
        if (id == 0) return;
        int posicao = buscarEquip(s, id);
        if (posicao == -1) {
            printf("  Equipamento nao encontrado.\n");
            continue;
```

```
        }
        int livres = unidadesLivres(s, &s->equips[posicao], r->data, r->turno, idIgnorado);
        if (livres == 0) {
            printf("  Nao ha unidades livres nesse turno.\n");
            continue;
        }
        while (indice < r->qtdItens && r->itens[indice].idEquipamento != id) {
            indice++;
        }
        sprintf(pergunta, "Quantidade (1 a %d): ", livres);
        r->itens[indice].idEquipamento = id;
        r->itens[indice].quantidade = lerInteiro(pergunta, 1, livres);
        if (indice == r->qtdItens) r->qtdItens++;
    }
    printf("Limite de %d tipos de equipamento atingido.\n", MAX_ITENS);
}

/* Alteracoes podem ficar temporariamente inconsistentes; valida-se ao confirmar. */
void alterarDadosReserva(Sistema *s, Reserva *r, int idIgnorado, int *semanas) {
    imprimirTitulo("Alterar dados");
    printf("1 - Data\n2 - Turno\n3 - Quantidade de alunos\n4 - Laboratorio\n");
    printf("5 - Professor e matricula\n6 - Disciplina\n7 - Equipamentos\n");
    if (semanas != NULL) printf("8 - Repeticao semanal\n");
    printf("0 - Voltar ao resumo\n");
    int opcao = lerInteiro("Opcao: ", 0, semanas == NULL ? 7 : 8);

    switch (opcao) {
    case 1:
        lerDataFutura(&r->data);
        break;
    case 2: {
        int turno = lerTurno();
        if (turno != 0) r->turno = turno;
        break;
    }
    case 3:
        r->qtdAlunos = lerInteiro("Quantidade de alunos: ", 1, MAX_ALUNOS);
        break;
    case 4: {
        int labAnterior = r->idLaboratorio;
        if (labsDisponiveis(s, r->data, r->turno, r->qtdAlunos, idIgnorado) == 0) {
            explicarFaltaLab(s, r->data, r->turno, r->qtdAlunos, idIgnorado);
            break;
        }
        listarLabsLivres(s, r->data, r->turno, r->qtdAlunos, idIgnorado);
        if (!escolherLab(s, r, idIgnorado)) r->idLaboratorio = labAnterior;
        break;
    }
    case 5:
        lerTexto("Nome do professor: ", r->professor, TAM_NOME);
        lerTexto("Matricula: ", r->matricula, TAM_CURTO);
        break;
    case 6:
        lerTexto("Disciplina: ", r->disciplina, TAM_NOME);
        break;
    case 7:
        selecionarEquips(s, r, idIgnorado);
        break;
    case 8:
        if (semanas != NULL)
            *semanas = lerInteiro("Semanas seguintes (0 para nenhuma): ", 0, MAX_SEMANAS_EXTRAS);
        break;
    default:
        return;
```

```
    }
    /* Uma alteracao isolada pode conflitar; as demais ainda poderao corrigi-la. */
}

int semanaDisponivel(Sistema *s, Reserva *r, int mostrarMotivo) {
    int posLab = buscarLab(s, r->idLaboratorio);
    if (!dataValida(r->data)) {
        if (mostrarMotivo) printf("data fora do limite permitido");
        return 0;
    }
    if (posLab == -1 || s->labs[posLab].capacidade < r->qtdAlunos ||
        conflitoLab(s, r->idLaboratorio, r->data, r->turno, 0) != -1) {
        if (mostrarMotivo) printf("laboratorio indisponivel");
        return 0;
    }
    for (int i = 0; i < r->qtdItens; i++) {
        int posEquip = buscarEquip(s, r->itens[i].idEquipamento);
        if (posEquip == -1 ||
            unidadesLivres(s, &s->equips[posEquip], r->data, r->turno, 0) <
            r->itens[i].quantidade) {
            if (mostrarMotivo) printf("equipamentos insuficientes");
            return 0;
        }
    }
    return 1;
}

int mostrarSemanas(Sistema *s, Reserva base, int semanas) {
    int aceitas = 0;

    printf("\nDatas da reserva:\n");
    printf("%02d/%02d/%04d - Reserva original: ",
           base.data.dia, base.data.mes, base.data.ano);
    if (!dataJaPassou(base.data) && semanaDisponivel(s, &base, 0)) {
        printf("Disponivel\n");
        aceitas++;
    } else {
        semanaDisponivel(s, &base, 1);
        printf(" (corrija antes de confirmar)\n");
    }
    for (int i = 1; i <= semanas; i++) {
        Reserva copia = base;
        copia.data = somarDias(base.data, 7 * i);
        printf("%02d/%02d/%04d - ", copia.data.dia, copia.data.mes, copia.data.ano);
        if (semanaDisponivel(s, &copia, 1)) {
            printf("Disponivel\n");
            aceitas++;
        } else {
            printf(" (ignorada)\n");
        }
    }
    printf("%d de %d datas disponiveis.\n", aceitas, semanas + 1);
    return aceitas;
}

void criarReserva(Sistema *s) {
    Reserva nova = {0};
    int semanas, opcao;

    if (s->info.qtdLabs == 0) {
        printf("Cadastre um laboratorio antes de fazer reservas.\n");
        return;
    }
    imprimirTitulo("Nova reserva");
```

```
    if (!configurarDisponibilidade(s, &nova, 0)) {
        printf("Cadastro interrompido.\n");
        return;
    }
    imprimirTitulo("Dados do responsavel");
    lerTexto("Nome do professor: ", nova.professor, TAM_NOME);
    lerTexto("Matricula: ", nova.matricula, TAM_CURTO);
    lerTexto("Disciplina: ", nova.disciplina, TAM_NOME);
    selecionarEquips(s, &nova, 0);

    semanas = lerInteiro("Repetir nas proximas quantas semanas (0 para nenhuma)? ",
                         0, MAX_SEMANAS_EXTRAS);
    while (1) {
        imprimirTitulo("Conferir reserva");
        exibirDetalhes(s, &nova);
        mostrarSemanas(s, nova, semanas);
        printf("\n1 - Confirmar\n2 - Alterar dados\n0 - Cancelar\n");
        opcao = lerInteiro("Opcao: ", 0, 2);
        if (opcao == 0) {
            printf("Reserva descartada.\n");
            return;
        }
        if (opcao == 2) {
            alterarDadosReserva(s, &nova, 0, &semanas);
            continue;
        }
        if (!validarReserva(s, &nova, 0)) {
            printf("Corrija os problemas antes de confirmar.\n");
            continue;
        }

        int qtdAnterior = s->info.qtdReservas;
        int proximoAnterior = s->info.proximoReserva;
        adicionarReserva(s, nova);
        for (int i = 1; i <= semanas; i++) {
            Reserva copia = nova;
            copia.data = somarDias(nova.data, 7 * i);
            if (semanaDisponivel(s, &copia, 0)) {
                adicionarReserva(s, copia);
            }
        }
        int criadas = s->info.qtdReservas - qtdAnterior;
        if (!salvarDados(s)) {
            desfazerInclusaoReservas(s, qtdAnterior, proximoAnterior);
            printf("Nenhuma reserva foi gravada. Tente novamente.\n");
            continue;
        }
        printf("%d reserva(s) confirmada(s):\n", criadas);
        for (int i = qtdAnterior; i < s->info.qtdReservas; i++) {
            Reserva *confirmada = &s->reservas[i];
            printf("  Reserva %d - %02d/%02d/%04d (%s)\n",
                   confirmada->id, confirmada->data.dia, confirmada->data.mes,
                   confirmada->data.ano, nomeTurno(confirmada->turno));
        }
        return;
    }
}

void consultarReserva(Sistema *s) {
    int posicao = buscarReserva(s, lerInteiro("ID da reserva: ", 1, MAX_ID));

    if (posicao == -1) {
        printf("Reserva nao encontrada.\n");
        return;
```

```
    }
    imprimirTitulo("Detalhes da reserva");
    exibirDetalhes(s, &s->reservas[posicao]);
}

/* Só permite editar reservas de datas que ainda não passaram. */
int escolherAlteravel(Sistema *s) {
    int posicao = buscarReserva(s, lerInteiro("ID da reserva: ", 1, MAX_ID));
    if (posicao == -1) {
        printf("Reserva nao encontrada.\n");
        return -1;
    }
    if (dataJaPassou(s->reservas[posicao].data)) {
        printf("Esta reserva ja ocorreu.\n");
        return -1;
    }
    return posicao;
}

void editarReserva(Sistema *s) {
    int posicao = escolherAlteravel(s);
    Reserva alterada, original;

    if (posicao == -1) {
        return;
    }
    alterada = s->reservas[posicao];
    while (1) {
        imprimirTitulo("Editar reserva");
        exibirDetalhes(s, &alterada);
        printf("\n1 - Salvar alteracoes\n2 - Alterar dados\n0 - Descartar\n");
        int opcao = lerInteiro("Opcao: ", 0, 2);
        if (opcao == 0) {
            printf("Alteracoes descartadas.\n");
            return;
        }
        if (opcao == 2) {
            alterarDadosReserva(s, &alterada, alterada.id, NULL);
            continue;
        }
        if (!validarReserva(s, &alterada, alterada.id)) {
            printf("Corrija os problemas antes de salvar.\n");
            continue;
        }
        original = s->reservas[posicao];
        s->reservas[posicao] = alterada;
        if (salvarDados(s)) {
            printf("Reserva atualizada.\n");
            return;
        }
        s->reservas[posicao] = original;
        printf("Alteracoes nao salvas. Tente novamente ou descarte.\n");
    }
}



void excluirReserva(Sistema *s) {
    int posicao = buscarReserva(s, lerInteiro("ID da reserva: ", 1, MAX_ID));
    Reserva removida;

    if (posicao == -1) {
        printf("Reserva nao encontrada.\n");
        return;
```

```
    }
    exibirDetalhes(s, &s->reservas[posicao]);
    if (!lerConfirmacao("Excluir definitivamente esta reserva?")) {
        return;
    }
    removida = removerReserva(s, posicao);
    if (salvarDados(s)) {
        printf("Reserva excluida.\n");
    } else {
        restaurarReserva(s, posicao, removida);
        printf("Exclusao nao realizada.\n");
    }
}

/* CRUD de laboratórios e equipamentos */

void criarLab(Sistema *s) {
    Laboratorio novo = {0};

    imprimirTitulo("Novo laboratorio");
    lerTexto("Nome: ", novo.nome, TAM_NOME);
    lerTexto("Bloco: ", novo.bloco, TAM_CURTO);
    lerTexto("Tipo: ", novo.tipo, TAM_NOME);
    novo.capacidade = lerInteiro("Capacidade de alunos: ", 1, MAX_ALUNOS);
    adicionarLab(s, novo);
    if (salvarDados(s)) {
        printf("Laboratorio cadastrado.\n");
    } else {
        s->info.qtdLabs--;
        s->info.proximoLab--;
        printf("Cadastro nao realizado.\n");
    }
}

/* A capacidade não pode ficar menor que a maior turma com reserva futura. */
void editarLab(Sistema *s) {
    int posicao = buscarLab(s, lerInteiro("ID do laboratorio: ", 1, MAX_ID));
    int maiorTurma = 0;
    int minimo = 1;
    Laboratorio *l, original;

    if (posicao == -1) {
        printf("Laboratorio nao encontrado.\n");
        return;
    }
    l = &s->labs[posicao];
    original = *l;

    for (int i = 0; i < s->info.qtdReservas; i++) {
        Reserva *r = &s->reservas[i];

        if (r->idLaboratorio == l->id && reservaFutura(r) && r->qtdAlunos > maiorTurma) {
            maiorTurma = r->qtdAlunos;
        }
    }
    lerTexto("Novo nome: ", l->nome, TAM_NOME);
    lerTexto("Novo bloco: ", l->bloco, TAM_CURTO);
    lerTexto("Novo tipo: ", l->tipo, TAM_NOME);
    if (maiorTurma > 0) {
        printf("  Existem reservas futuras com ate %d alunos.\n", maiorTurma);
        minimo = maiorTurma;
    }
    l->capacidade = lerInteiro("Nova capacidade: ", minimo, MAX_ALUNOS);
    if (salvarDados(s)) {
```

```
        printf("Laboratorio atualizado.\n");
    } else {
        *l = original;
        printf("Alteracoes nao realizadas.\n");
    }
}

void excluirLab(Sistema *s) {
    int posicao = buscarLab(s, lerInteiro("ID do laboratorio: ", 1, MAX_ID));
    Laboratorio removido;

    if (posicao == -1) {
        printf("Laboratorio nao encontrado.\n");
        return;
    }
    for (int i = 0; i < s->info.qtdReservas; i++) {
        if (s->reservas[i].idLaboratorio == s->labs[posicao].id &&
            reservaFutura(&s->reservas[i])) {
            printf("Nao e possivel excluir: ha reservas futuras neste laboratorio.\n");
            return;
        }
    }
    if (!lerConfirmacao("Excluir este laboratorio?")) {
        return;
    }
    removido = removerLab(s, posicao);
    if (salvarDados(s)) {
        printf("Laboratorio excluido.\n");
    } else {
        restaurarLab(s, posicao, removido);
        printf("Exclusao nao realizada.\n");
    }
}

void criarEquip(Sistema *s) {
    Equipamento novo = {0};

    imprimirTitulo("Novo equipamento");
    lerTexto("Nome: ", novo.nome, TAM_NOME);
    lerTexto("Categoria: ", novo.categoria, TAM_NOME);
    novo.quantidadeTotal = lerInteiro("Quantidade total: ", 1, MAX_ESTOQUE);
    adicionarEquip(s, novo);
    if (salvarDados(s)) {
        printf("Equipamento cadastrado.\n");
    } else {
        s->info.qtdEquips--;
        s->info.proximoEquip--;
        printf("Cadastro nao realizado.\n");
    }
}

/* O estoque total não pode ser menor que o uso de um turno já reservado. */
void editarEquip(Sistema *s) {
    int posicao = buscarEquip(s, lerInteiro("ID do equipamento: ", 1, MAX_ID));
    int maiorUso = 0;
    int minimo = 1;
    Equipamento *e, original;

    if (posicao == -1) {
        printf("Equipamento nao encontrado.\n");
        return;
    }
    e = &s->equips[posicao];
    original = *e;
```

```
    for (int i = 0; i < s->info.qtdReservas; i++) {
        Reserva *r = &s->reservas[i];

        if (reservaFutura(r) && qtdNaReserva(r, e->id) > 0) {
            int uso = unidadesEmUso(s, e->id, r->data, r->turno, 0);

            if (uso > maiorUso) {
                maiorUso = uso;
            }
        }
    }
    lerTexto("Novo nome: ", e->nome, TAM_NOME);
    lerTexto("Nova categoria: ", e->categoria, TAM_NOME);
    if (maiorUso > 0) {
        printf("  Reservas futuras usam ate %d unidades ao mesmo tempo.\n", maiorUso);
        minimo = maiorUso;
    }
    e->quantidadeTotal = lerInteiro("Nova quantidade total: ", minimo, MAX_ESTOQUE);
    if (salvarDados(s)) {
        printf("Equipamento atualizado.\n");
    } else {
        *e = original;
        printf("Alteracoes nao realizadas.\n");
    }
}

void excluirEquip(Sistema *s) {
    int posicao = buscarEquip(s, lerInteiro("ID do equipamento: ", 1, MAX_ID));
    Equipamento removido;

    if (posicao == -1) {
        printf("Equipamento nao encontrado.\n");
        return;
    }
    for (int i = 0; i < s->info.qtdReservas; i++) {
        if (reservaFutura(&s->reservas[i]) &&
            qtdNaReserva(&s->reservas[i], s->equips[posicao].id) > 0) {
            printf("Nao e possivel excluir: ha reservas futuras com este equipamento.\n");
            return;
        }
    }
    if (!lerConfirmacao("Excluir este equipamento?")) {
        return;
    }
    removido = removerEquip(s, posicao);
    if (salvarDados(s)) {
        printf("Equipamento excluido.\n");
    } else {
        restaurarEquip(s, posicao, removido);
        printf("Exclusao nao realizada.\n");
    }
}

/* Consultas e relatórios */

void listarTodasReservas(Sistema *s) {
    Data semData = {0, 0, 0};

    listarReservas(s, "Todas as reservas", FILTRO_TODAS, semData, "");
}

void exibirAgendaDoDia(Sistema *s) {
    Data d;
```

```
    char titulo[64];

    lerData("Data (DD/MM/AAAA): ", &d);
    sprintf(titulo, "Agenda de %s, %02d/%02d/%04d", nomeDiaSemana(d), d.dia, d.mes, d.ano);
    imprimirTitulo(titulo);
    for (int turno = TURNO_MANHA; turno <= TURNO_NOITE; turno++) {
        int achadas = 0;
        imprimirTitulo(nomeTurno(turno));
        for (int i = 0; i < s->info.qtdReservas; i++) {
            Reserva *r = &s->reservas[i];
            if (dataInt(r->data) == dataInt(d) &&
                r->turno == turno) {
                printf("  %-22.22s - %-20.20s (%s, %d alunos)\n",
                       nomeLab(s, r->idLaboratorio), r->professor,
                       r->disciplina, r->qtdAlunos);
                achadas++;
            }
        }
        if (achadas == 0) {
            printf("  Nenhuma reserva.\n");
        }
    }
}

void buscarPorProfessor(Sistema *s) {
    char procurado[TAM_NOME];
    Data semData = {0, 0, 0};

    lerTexto("Nome ou matricula: ", procurado, TAM_NOME);
    listarReservas(s, "Reservas encontradas", FILTRO_PROFESSOR, semData, procurado);
}

void consultarDisponibilidade(Sistema *s) {
    Data d;
    int turno;

    lerData("Data (DD/MM/AAAA): ", &d);
    turno = lerTurno();
    if (turno == 0) {
        return;
    }
    printf("Data: %02d/%02d/%04d - %s\n", d.dia, d.mes, d.ano, nomeTurno(turno));
    listarLabsLivres(s, d, turno, 0, 0);
    listarEquipsLivres(s, d, turno, 0);
}

void exibirRelatorio(Sistema *s) {
    int *turnos = calloc((size_t)s->info.qtdLabs + 1, sizeof(int));
    int *unidades = calloc((size_t)s->info.qtdEquips + 1, sizeof(int));

    if (turnos == NULL || unidades == NULL) {
        printf("Memoria insuficiente para gerar o relatorio.\n");
        free(turnos);
        free(unidades);
        return;
    }
    for (int i = 0; i < s->info.qtdReservas; i++) {
        Reserva *r = &s->reservas[i];

        int posLab = buscarLab(s, r->idLaboratorio);
        if (posLab != -1) {
            turnos[posLab]++;
        }
        for (int j = 0; j < r->qtdItens; j++) {
```

```
            int posEquip = buscarEquip(s, r->itens[j].idEquipamento);
            if (posEquip != -1) {
                unidades[posEquip] += r->itens[j].quantidade;
            }
        }
    }

    imprimirTitulo("Relatorio de uso");
    printf("Total de reservas: %d\n", s->info.qtdReservas);

    imprimirTitulo("Turnos reservados por laboratorio");
    for (int i = 0; i < s->info.qtdLabs; i++) {
        printf("%-22.22s %d turnos\n", s->labs[i].nome, turnos[i]);
    }
    imprimirTitulo("Total de unidades solicitadas por equipamento");
    for (int i = 0; i < s->info.qtdEquips; i++) {
        printf("%-22.22s %d unidades\n", s->equips[i].nome, unidades[i]);
    }
    free(turnos);
    free(unidades);
}

/* Menus */

void menuReservas(Sistema *s) {
    while (1) {
        imprimirTitulo("Reservas");
        printf("1 - Nova reserva\n");
        printf("2 - Listar reservas\n");
        printf("3 - Consultar reserva\n");
        printf("4 - Editar reserva\n");
        printf("5 - Excluir reserva\n");
        printf("0 - Voltar\n");

        switch (lerInteiro("Opcao: ", 0, 5)) {
        case 1: criarReserva(s); break;
        case 2: listarTodasReservas(s); break;
        case 3: consultarReserva(s); break;
        case 4: editarReserva(s); break;
        case 5: excluirReserva(s); break;
        default: return;
        }
    }
}

void menuLabs(Sistema *s) {
    while (1) {
        imprimirTitulo("Laboratorios");
        printf("1 - Cadastrar\n");
        printf("2 - Listar\n");
        printf("3 - Editar\n");
        printf("4 - Excluir\n");
        printf("0 - Voltar\n");

        switch (lerInteiro("Opcao: ", 0, 4)) {
        case 1: criarLab(s); break;
        case 2: listarLabs(s); break;
        case 3: editarLab(s); break;
        case 4: excluirLab(s); break;
        default: return;
        }
    }
}
```

```
void menuEquips(Sistema *s) {
    while (1) {
        imprimirTitulo("Equipamentos");
        printf("1 - Cadastrar\n");
        printf("2 - Listar\n");
        printf("3 - Editar\n");
        printf("4 - Excluir\n");
        printf("0 - Voltar\n");

        switch (lerInteiro("Opcao: ", 0, 4)) {
        case 1: criarEquip(s); break;
        case 2: listarEquips(s); break;
        case 3: editarEquip(s); break;
        case 4: excluirEquip(s); break;
        default: return;
        }
    }
}

void menuConsultas(Sistema *s) {
    while (1) {
        imprimirTitulo("Consultas e relatorios");
        printf("1 - Agenda do dia\n");
        printf("2 - Buscar por professor\n");
        printf("3 - Consultar disponibilidade\n");
        printf("4 - Relatorio de uso\n");
        printf("0 - Voltar\n");

        switch (lerInteiro("Opcao: ", 0, 4)) {
        case 1: exibirAgendaDoDia(s); break;
        case 2: buscarPorProfessor(s); break;
        case 3: consultarDisponibilidade(s); break;
        case 4: exibirRelatorio(s); break;
        default: return;
        }
    }
}

int main(void) {
    Sistema s = {0};
    int executando = 1;

    s.info.assinatura = ASSINATURA_ARQUIVO;
    s.info.proximoLab = 1;
    s.info.proximoEquip = 1;
    s.info.proximoReserva = 1;

    if (!carregarDados(&s)) {
        cadastrarDadosIniciais(&s);
        if (!salvarDados(&s)) {
            free(s.labs);
            free(s.equips);
            return 1;
        }
    }

    printf("============================================\n");
    printf("  LabReserva - Reserva de laboratorios\n");
    printf("============================================\n");

    while (executando) {
        imprimirTitulo("Menu principal");
        printf("1 - Reservas\n");
        printf("2 - Laboratorios\n");
```

```
        printf("3 - Equipamentos\n");
        printf("4 - Consultas e relatorios\n");
        printf("0 - Sair\n");

        switch (lerInteiro("Opcao: ", 0, 4)) {
        case 1: menuReservas(&s); break;
        case 2: menuLabs(&s); break;
        case 3: menuEquips(&s); break;
        case 4: menuConsultas(&s); break;
        default: executando = 0; break;
        }
    }

    free(s.labs);
    free(s.equips);
    free(s.reservas);
    printf("Ate logo!\n");
    return 0;
}
```